

TD06 : Architecture MVC Avancée (2/2)

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	13

1) Problématique

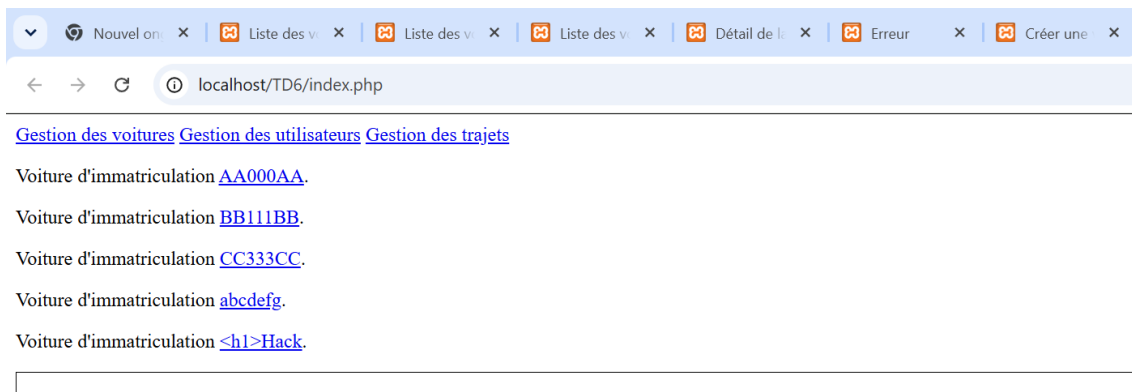
Dans ce TD, on veut améliorer notre site en MVC pour éviter de répéter tout le temps le même code. Jusqu'à maintenant, on réécrivait les mêmes requêtes SQL et les mêmes fonctions pour chaque table (voiture, utilisateur, trajet). On va donc créer une classe mère Model avec des méthodes génériques pour faire le select, le save, le delete et le update directement grâce à l'héritage. On va aussi adapter notre routeur pour qu'il puisse gérer plusieurs contrôleurs et vérifier les actions demandées, et on va fusionner les formulaires de création et de modification en une seule page pour rendre notre code plus simple et plus propre.

2) Exercices

Exercice 1 : Action par défaut du routeur (readAll)

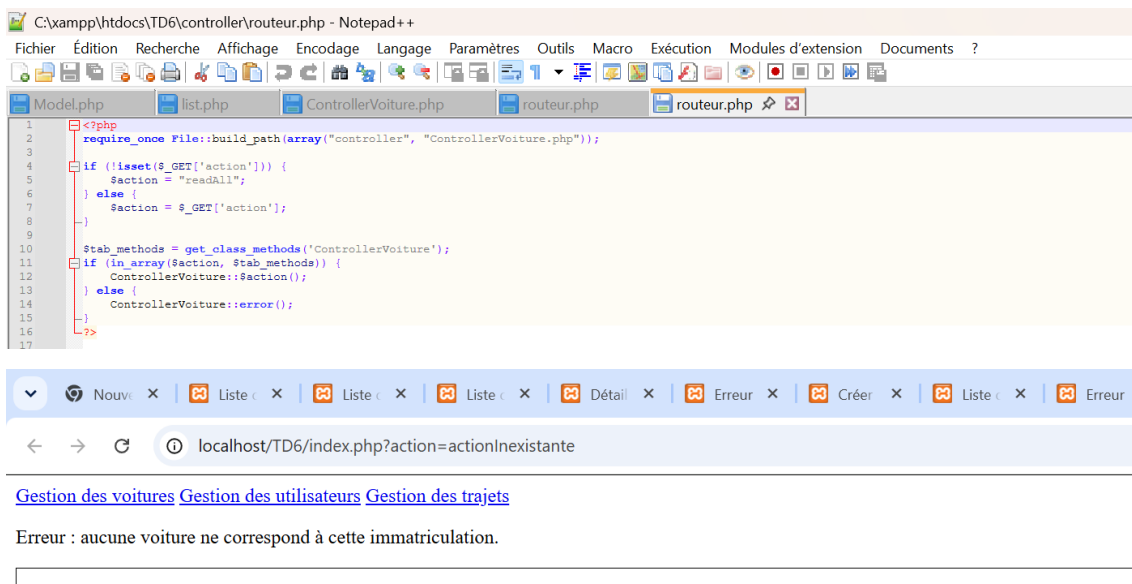
Dans le routeur (routeur.php), si aucune action n'est précisée dans l'URL avec \$_GET['action'], on met par défaut 'readAll' dans la variable \$action. Comme ça, quand on va sur index.php, ça affiche directement la liste de toutes les voitures sans faire d'erreur PHP.

```
1 <?php
2 require_once File::build_path(array("controller", "ControllerVoiture.php"));
3
4 if (!isset($_GET['action'])) {
5     $action = "readAll";
6 } else {
7     $action = $_GET['action'];
8 }
9
10 ControllerVoiture::readAll();
11
12 >>
```



Exercice 2 : Vérification de l'action dans le contrôleur

Pour éviter les bugs si quelqu'un tape une mauvaise action dans l'URL, on vérifie que l'action demandée existe bien dans la classe ControllerVoiture. On utilise la fonction `get_class_methods('ControllerVoiture')` pour avoir la liste des méthodes du contrôleur, puis `in_array($action, $tab_methods)` pour voir si notre action est dedans. Si elle n'existe pas, on appelle `ControllerVoiture::error()` pour afficher la vue d'erreur.



Exercice 3 : Action delete dans ControllerVoiture et ModelVoiture

Pour supprimer une voiture, on crée d'abord `deleteByImmat($immat)` dans `ModelVoiture.php` avec une requête SQL DELETE. Dans `ControllerVoiture.php`, on fait la méthode `delete()` qui prend l'immatriculation reçue en GET, appelle la suppression dans le modèle, recharge la liste des voitures avec `ModelVoiture::selectAll()` et affiche la vue `deleted.php`. On rajoute aussi un lien pour supprimer la voiture dans la vue `detail.php`.

C:\xampp\htdocs\TD6\model\ModelVoiture.php - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

Model.php list.php ControllerVoiture.php routeur.php ModelVoiture.php

```
1 <?php
2 require_once File::build_path(array("model", "Model.php"));
3
4 class ModelVoiture {
5     private $immatriculation;
6     private $marque;
7     private $couleur;
8
9     public function __construct($m = NULL, $c = NULL, $i = NULL) {
10         if (!is_null($m) && !is_null($c) && !is_null($i)) {
11             $this->marque = $m;
12             $this->couleur = $c;
13             $this->immatriculation = $i;
14         }
15     }
16
17     public function getImmatriculation() {
18         return $this->immatriculation;
19     }
20 }
```

Nouv Liste Liste Liste Détail Erreur Créer Liste Erreur Erreur Détail

localhost/TD6/index.php?action=read&immat=DEL123

[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

Voiture DEL123 de marque Citroen (couleur : Verte)

[Supprimer cette voiture](#)

Nouv Liste Liste Liste Détail Erreur Créer Liste Erreur Erreur Détail Supp

localhost/TD6/index.php?action=delete&immat=DEL123

[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

La voiture d'immatriculation DEL123 a bien été supprimée !

Voiture d'immatriculation [AA000AA](#).

Voiture d'immatriculation [BB111BB](#).

Voiture d'immatriculation [CC333CC](#).

Voiture d'immatriculation [abcdefg](#).

Voiture d'immatriculation [<h1>Hack](#).

Exercice 4 : Action update (formulaire pré-rempli en lecture seule)

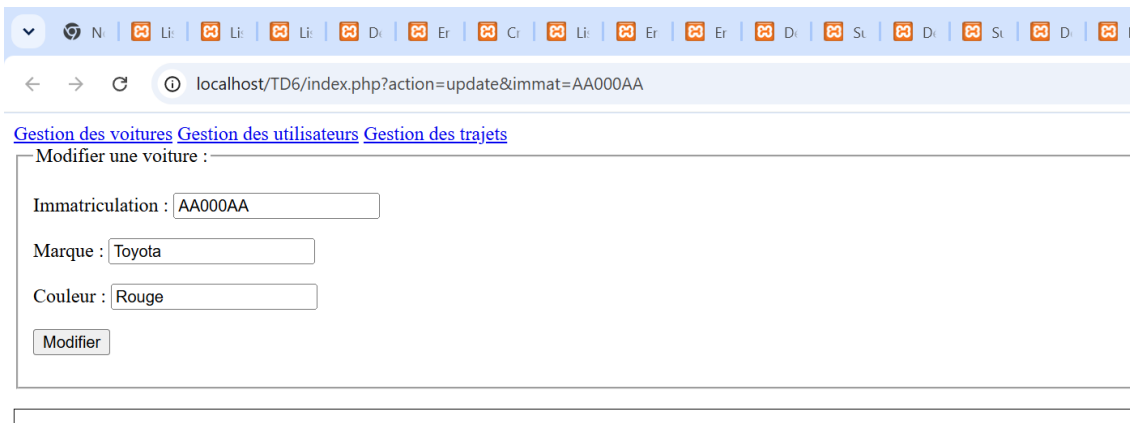
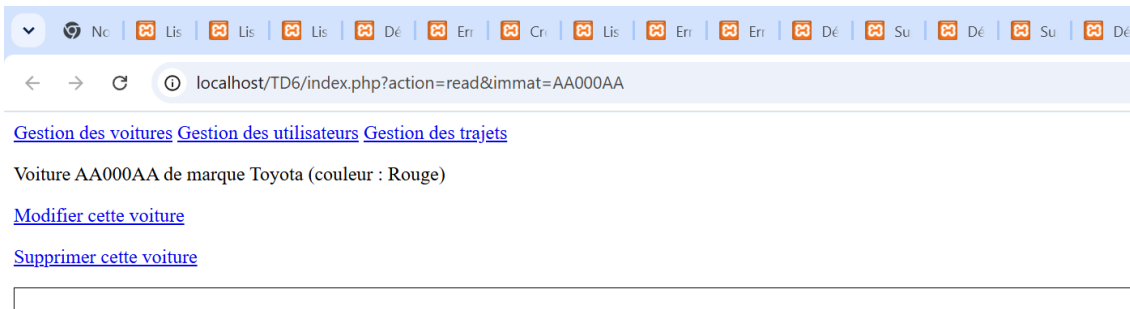
Pour modifier une voiture, on crée le formulaire dans view/voiture/update.php. On remplit les champs avec les informations actuelles de la voiture. Comme l'immatriculation est la clé primaire et qu'on ne doit pas pouvoir la changer, on met le champ en readonly. Dans ControllerVoiture.php, on fait la méthode update() et on ajoute un lien 'Modifier cette voiture' dans la vue detail.php.

C:\xampp\htdocs\TD6\view\voiture\update.php - Notepad++

Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

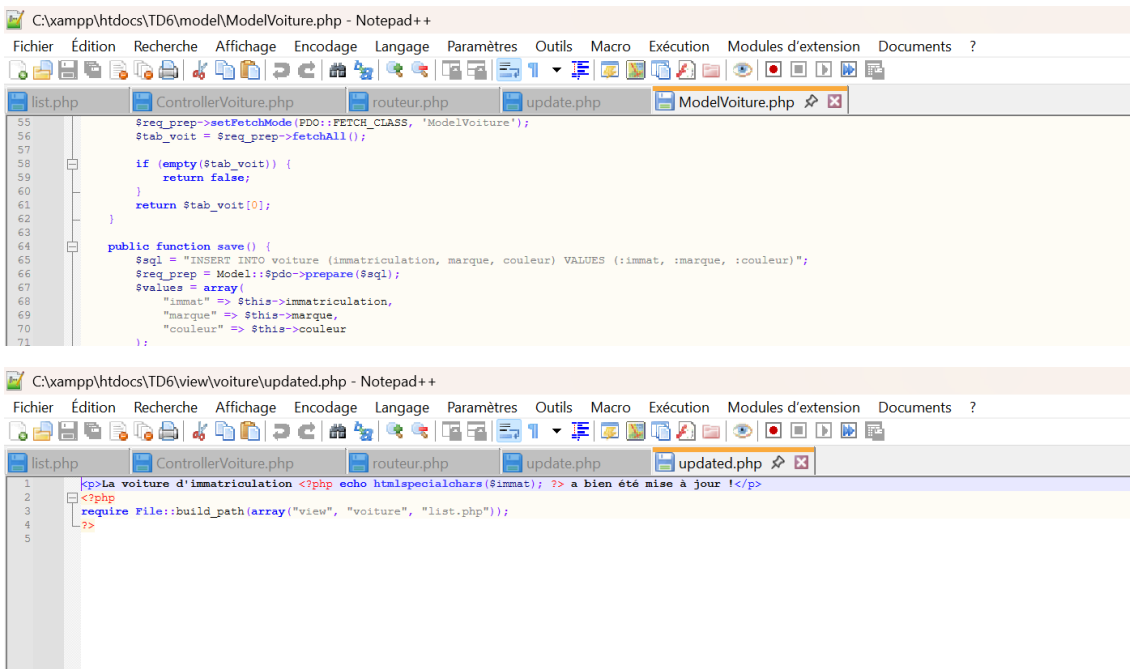
list.php ControllerVoiture.php routeur.php update.php

```
4 <input type="hidden" name="action" value="updated">
5 <p>
6     <label for="immat_id">Immatriculation :</label>
7     <input type="text" value="<?php echo htmlspecialchars($v->getImmatriculation()); ?>" name="immat" id="immat_id" readonly />
8 </p>
9
10 <label for="marque_id">Marque :</label>
11 <input type="text" value="<?php echo htmlspecialchars($v->getMarque()); ?>" name="marque" id="marque_id" required />
12 </p>
13
14 <label for="couleur_id">Couleur :</label>
15 <input type="text" value="<?php echo htmlspecialchars($v->getCouleur()); ?>" name="couleur" id="couleur_id" required />
16 </p>
17
18 <input type="submit" value="Modifier" />
19 </p>
20 </fieldset>
```



Exercice 5 : Action updated (mise à jour en base de données)

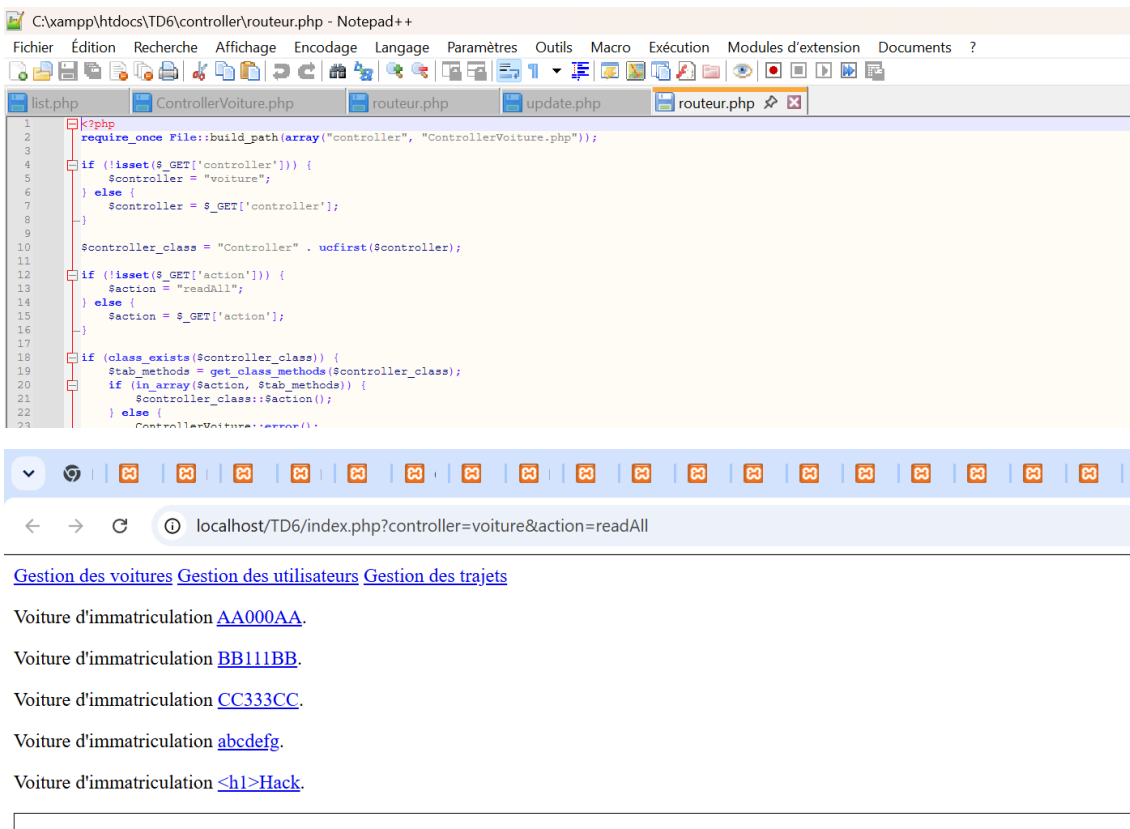
Pour enregistrer les modifications, on ajoute la méthode `update($data)` dans `ModelVoiture.php` qui fait un UPDATE en SQL avec une requête préparée. Dans `ControllerVoiture.php`, l'action `updated()` prend les données envoyées, appelle la mise à jour dans le modèle et affiche la vue `updated.php` pour confirmer que la voiture a bien été modifiée avant de réafficher la liste.





Exercice 6 : Gérer plusieurs contrôleurs via le routeur

Pour que le routeur puisse gérer d'autres pages que les voitures, on récupère le contrôleur passé dans l'URL avec \$_GET['controller'], ou 'voiture' par défaut. On crée le nom de la classe avec \$controller_class = 'Controller' . ucfirst(\$controller). On vérifie ensuite que la classe existe avec class_exists() avant d'appeler l'action demandée.



Exercice 7 : Début du contrôleur Utilisateur

On crée la classe ModelUtilisateur.php avec les attributs login, nom et prénom, le constructeur, les getters et les setters. On crée aussi ControllerUtilisateur.php avec l'action readAll() et sa vue view/utilisateur/list.php, puis on ajoute le contrôleur dans routeur.php.

Réponse à la question 2 : La différence entre ControllerUtilisateur et ControllerVoiture, c'est qu'on utilise ModelUtilisateur au lieu de ModelVoiture, on appelle getAllUtilisateurs(), on stocke les résultats dans \$tab_u au lieu de \$tab_v, et on utilise les vues du dossier view/utilisateur/ au lieu de view/voiture/.

Réponse à la question 5 : La différence entre ModelUtilisateur et ModelVoiture, c'est que les attributs sont différents (login, nom, prénom au lieu de immatriculation, marque, couleur), on interroge la table utilisateur au lieu de voiture, et les objets créés sont des ModelUtilisateur.

```
C:\xampp\htdocs\TD6\model\ModelUtilisateur.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

list.php ControllerVoiture.php routeur.php update.php ControllerUtilisateur.php ModelUtilisateur.php
1
2 require_once File::build_path(array("model", "Model.php"));
3
4 class ModelUtilisateur {
5     private $login;
6     private $nom;
7     private $prenom;
8
9     public function __construct($l = NULL, $n = NULL, $p = NULL) {
10         if (!is_null($l) && !is_null($n) && !is_null($p)) {
11             $this->login = $l;
12             $this->nom = $n;
13             $this->prenom = $p;
14         }
15     }
16
17     public function getLogin() {
18         return $this->login;
19     }
20 }
```

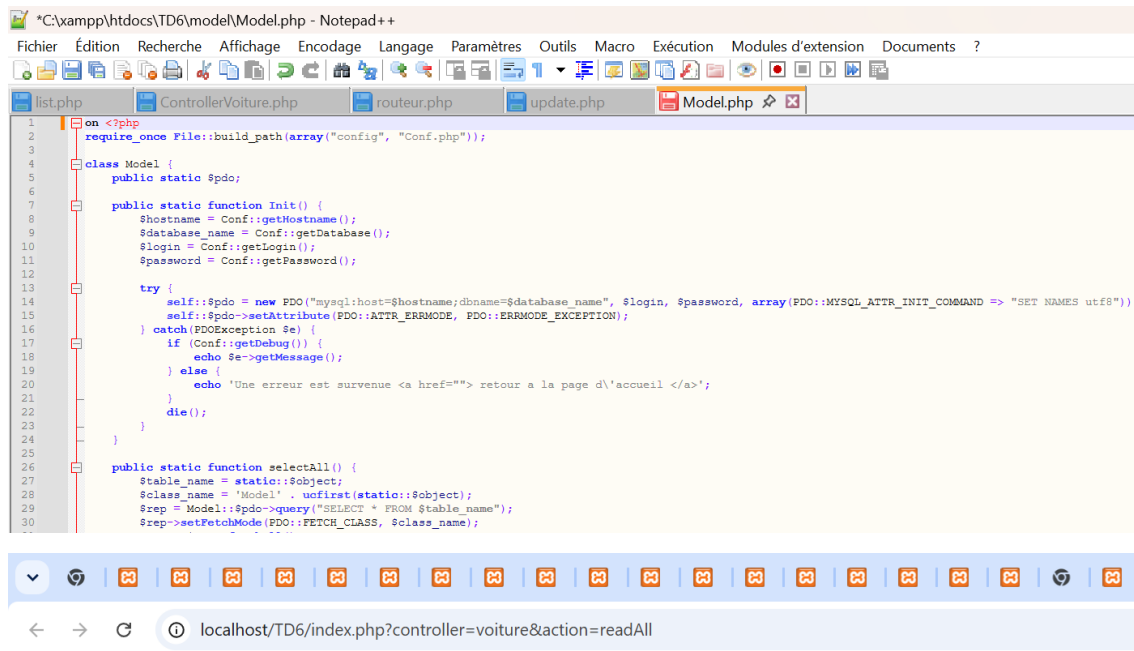
```
C:\xampp\htdocs\TD6\controller\ControllerUtilisateur.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

list.php ControllerVoiture.php routeur.php update.php ControllerUtilisateur.php
1
2 require_once File::build_path(array("model", "ModelUtilisateur.php"));
3
4 class ControllerUtilisateur {
5     public static function readAll() {
6         $tab_u = ModelUtilisateur::getAllUtilisateurs();
7         $controller = 'utilisateur';
8         $view = 'list';
9         $pagetitle = 'Liste des utilisateurs';
10        require File::build_path(array("view", "view.php"));
11    }
12 }
13
14 >>
```



Exercice 8 : Modèle générique avec selectAll() et static::\$object

Au lieu de réécrire getAll pour chaque modèle, on déplace la méthode dans Model.php et on l'appelle selectAll(). ModelVoiture et ModelUtilisateur héritent de Model avec le mot-clé extends. On ajoute une variable protected static \$object dans chaque modèle ('voiture' et 'utilisateur'). Grâce à static::\$object, la fonction selectAll() sait automatiquement quelle table appeler et quelle classe instancier avec 'Model' . ucfirst(static::\$object).



[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

Voiture d'immatriculation [AA000AA](#).

Voiture d'immatriculation [BB111BB](#).

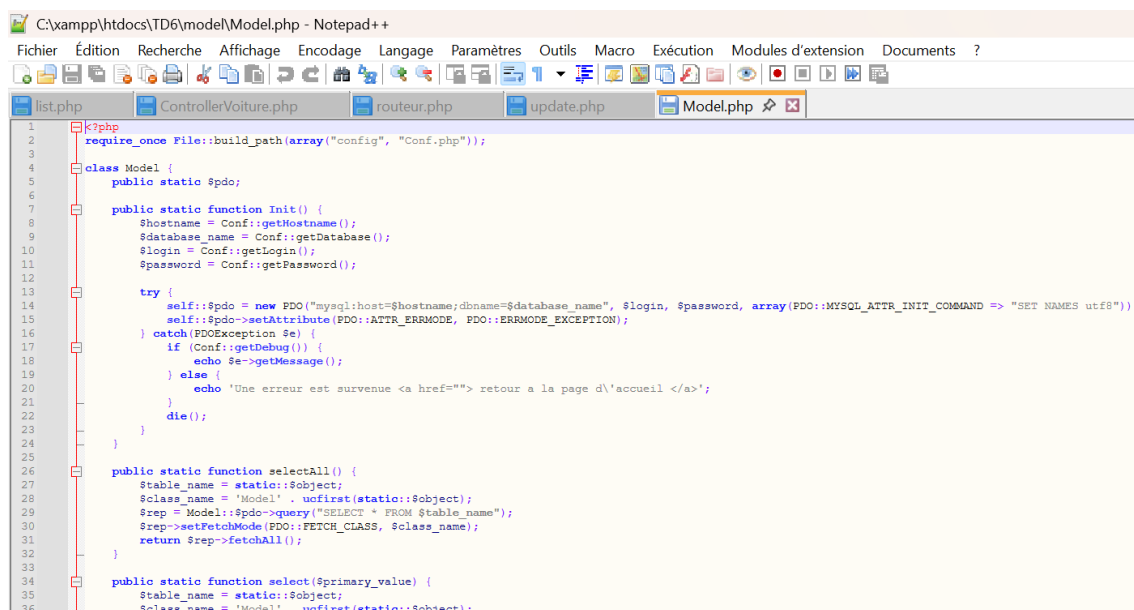
Voiture d'immatriculation [CC333CC](#).

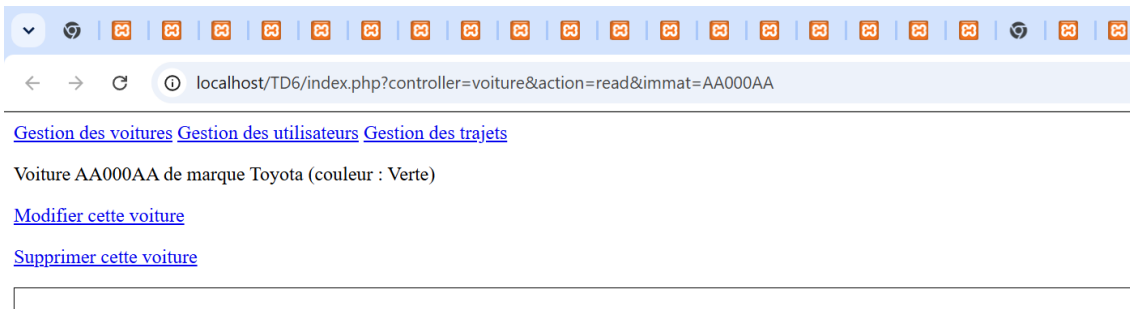
Voiture d'immatriculation [abcdefg](#).

Voiture d'immatriculation [<h1>Hack](#).

Exercice 9 : Méthode générique select() avec static::\$primary

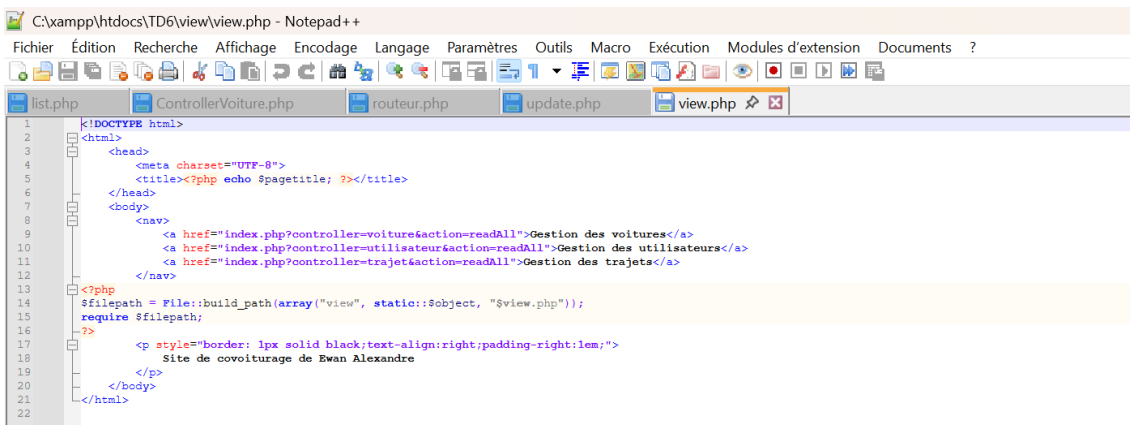
On fait la même chose pour chercher un enregistrement par sa clé primaire. Dans ModelVoiture et ModelUtilisateur, on ajoute la variable protected static \$primary ('immatriculation' pour la voiture et 'login' pour l'utilisateur). Dans Model.php, on écrit la méthode select(\$primary_value) avec la requête `SELECT * FROM $table_name WHERE $primary_key = '$primary_value'`. ControllerVoiture::read() peut alors directement appeler ModelVoiture::select(\$immat).





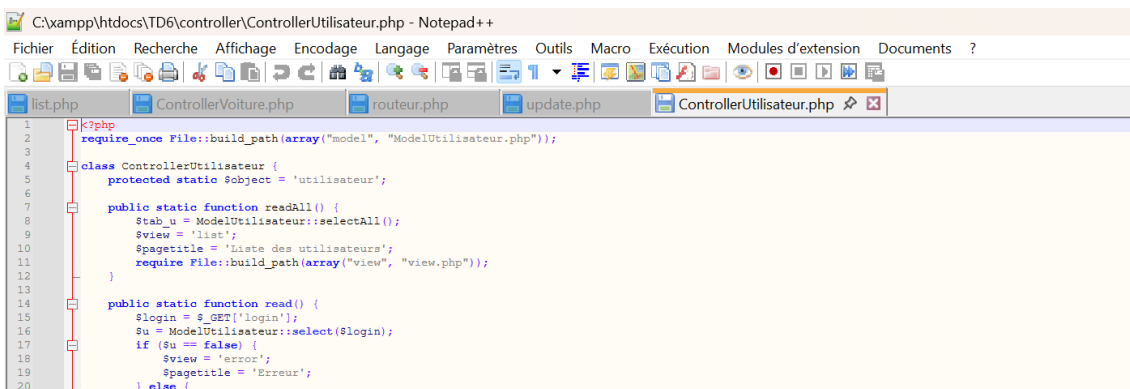
Exercice 10 : Utilisation de static::\$object dans la vue générique

Dans ControllerVoiture et ControllerUtilisateur, on ajoute un attribut protected static \$object. Dans view/view.php, au lieu de devoir passer une variable \$controller à chaque fois, on utilise directement static::\$object pour trouver le bon dossier de vue avec File::build_path(array('view', static::\$object, "\$view.php")).



Exercice 11 : Action read pour Utilisateur et vue detail.php

Dans ControllerUtilisateur, on crée l'action read() en appelant ModelUtilisateur::select(\$_GET['login']). On crée la vue view/utilisateur/detail.php pour afficher le login, le nom et le prénom, et on rajoute des liens vers la vue de détail dans la liste list.php.



localhost/TD6/index.php?controller=utilisateur&action=read&login=dupontj

[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

Utilisateur : dupontj

Nom : Dupont

Prénom : Jean

Exercice 12 : Méthode générique delete() dans Model.php

On crée une méthode générique `delete($primary_value)` dans `Model.php` qui fait la requête `DELETE` sur la bonne table avec sa clé primaire. Dans `ControllerUtilisateur.php`, on fait l'action `delete()`, on crée la vue `deleted.php` et on ajoute des liens pour supprimer un utilisateur dans la liste et dans la vue de détail.

Line 21: <code>}</code>
Line 22: <code>die();</code>
Line 23: <code>}</code>
Line 24: <code>}</code>
Line 25: <code></code>
Line 26: <code>public static function selectAll() {</code>
Line 27: <code> \$table_name = static::\$object;</code>
Line 28: <code> \$class_name = 'Model' . ucfirst(static::\$object);</code>
Line 29: <code> \$req = Model::\$pdo->query("SELECT * FROM \$table_name");</code>
Line 30: <code> \$req->setFetchMode(PDO::FETCH_CLASS, \$class_name);</code>
Line 31: <code> return \$req->fetchAll();</code>
Line 32: <code>}</code>
Line 33: <code></code>
Line 34: <code>public static function select(\$primary_value) {</code>
Line 35: <code> \$table_name = static::\$object;</code>
Line 36: <code> \$class_name = 'Model' . ucfirst(static::\$object);</code>
Line 37: <code> \$primary_key = static::\$primary;</code>
Line 38: <code></code>
Line 39: <code> \$sql = "SELECT * FROM \$table_name WHERE \$primary key = :taq";</code>

localhost/TD6/index.php?controller=utilisateur&action=readAll

[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

Utilisateur [dupontj](#) (Jean Dupont) [Supprimer](#)

Utilisateur [durandm](#) (Marie Durand) [Supprimer](#)

Utilisateur [martinluc](#) (Luc Martin) [Supprimer](#)

Utilisateur [suppr_user](#) (User Suppr) [Supprimer](#)

Utilisateur [usertest](#) (Testeur Test) [Supprimer](#)

localhost/TD6/index.php?controller=utilisateur&action=delete&login=suppr_user

[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

L'utilisateur de login `suppr_user` a bien été supprimé !

Utilisateur [dupontj](#) (Jean Dupont) [Supprimer](#)

Utilisateur [durandm](#) (Marie Durand) [Supprimer](#)

Utilisateur [martinluc](#) (Luc Martin) [Supprimer](#)

Utilisateur [usertest](#) (Testeur Test) [Supprimer](#)

Exercice 13 : Fusion des vues create.php et update.php pour Voiture

Comme les formulaires de création et de modification sont quasiment pareils, on supprime create.php pour garder seulement update.php. Dans ControllerVoiture, create() met les champs à vide, l'attribut à 'required' et l'action du formulaire à 'save'. L'action update() remplit les champs avec la voiture existante, met l'immatriculation en 'readonly' et l'action à 'updated'. Le formulaire s'adapte tout seul et on rajoute aussi un champ caché controller.

The image shows the source code of the `update.php` file in a Notepad++ editor. The code is a PHP script that uses the `htmlspecialchars` function to escape special characters in the form fields. It includes a `form` tag with `method="get"` and `action="index.php"`. The form contains a legend with two actions: 'Créer une voiture' and 'Modifier une voiture'. The 'Créer' action is associated with the `save` action, and the 'Modifier' action is associated with the `updated` action. The form fields are: 'Immatriculation' (text input), 'Marque' (text input), and 'Couleur' (text input). The 'Immatriculation' field is marked as 'required' and 'readonly'. The 'Marque' and 'Couleur' fields are marked as 'required'. The 'Créer' button is associated with the `save` action, and the 'Modifier' button is associated with the `updated` action.

Below the code, two browser screenshots are shown. The first screenshot shows the 'Créer une voiture' form with the URL `localhost/TD6/index.php?controller=voiture&action=create`. The second screenshot shows the 'Modifier une voiture' form with the URL `localhost/TD6/index.php?controller=voiture&action=update&immat=AA000AA`. The 'Modifier' form has pre-filled values for 'Immatriculation' (AA000AA), 'Marque' (Toyota), and 'Couleur' (Verte).

Exercice 14 : Adaptation des actions create et update pour Utilisateur

On fait pareil pour les utilisateurs : on crée les actions create() et update() dans ControllerUtilisateur.php, on fait la vue fusionnée view/utilisateur/update.php avec le champ login en required ou en readonly selon l'action, et on ajoute un lien pour modifier l'utilisateur dans detail.php.

```
*C:\xampp\htdocs\TD6\view\utilisateur\update.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

list.php ControllerVoiture.php routeur.php update.php update.php
1 <form method="get" action="index.php">
2
3 <div>
4 <input type="text" value="<?php echo htmlspecialchars($login); ?>" name="login" id="login_id" />
5 <input type="text" value="<?php echo htmlspecialchars($nom); ?>" name="nom" id="nom_id" required />
6 <input type="text" value="<?php echo htmlspecialchars($prenom); ?>" name="prenom" id="prenom_id" required />
7 <input type="submit" value="<?php echo ($action == 'save') ? 'Créer' : 'Modifier'; ?>" />
8
9 </div>
10
11 </form>
12
13
14
15
16
17
18
19
20
..
```



[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

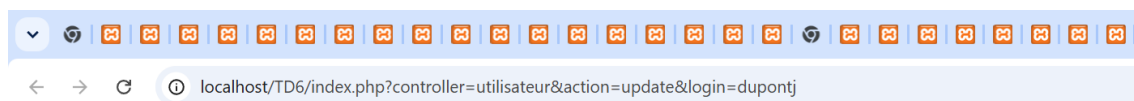
Utilisateur : dupontj

Nom : Dupont

Prénom : Jean

[Modifier cet utilisateur](#)

[Supprimer cet utilisateur](#)



[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

Modifier un utilisateur :

Login :

Nom :

Prénom :

Exercices 15 et 16 : Méthodes génériques update(\$data) et save(\$data) dans Model.php

Pour ne plus avoir à réécrire de requêtes SQL dans chaque modèle, on déplace update(\$data) et save(\$data) dans Model.php :

1. Dans update(\$data), on utilise une boucle foreach pour créer la partie SET champ=:champ pour chaque valeur, et on enlève la virgule en trop à la fin avec rtrim(). On ajoute ensuite le WHERE avec static::\$primary.
2. Dans save(\$data), on fait la même chose pour créer la liste des colonnes et des valeurs pour la requête INSERT INTO.

Comme ça, nos modèles n'ont plus du tout besoin de requêtes SQL, tout est géré directement par Model.php.

```
C:\xampp\htdocs\TD6\model\Model.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

ControllerVoiture.php routeur.php update.php update.php Model.php x
33
34 public static function select($primary_value) {
35     $table_name = static::$object;
36     $class_name = 'Model' . ucfirst(static::$object);
37     $primary_key = static::$primary;
38
39     $sql = "SELECT * FROM $table_name WHERE $primary_key = :tag";
40     $req = Model::pdo->prepare($sql);
41     $values = array(":tag" => $primary_value);
42     $req->execute($values);
43     $req->setFetchMode(PDO::FETCH_CLASS, $class_name);
44     $stab = $req->fetchAll();
45
46     if (empty($stab)) {
47         return false;
48     }
49     return $stab[0];
50 }
51
52 public static function delete($primary_value) {
53     $table_name = static::$object;
54     $class_name = 'Model' . ucfirst(static::$object);
55     $primary_key = static::$primary;
```



[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

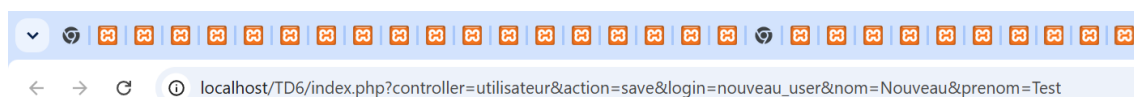
L'utilisateur de login martinluc a bien été mis à jour !

Utilisateur [dupontj](#) (Jean Dupont) [Supprimer](#)

Utilisateur [durandm](#) (Marie Durand) [Supprimer](#)

Utilisateur [martinluc](#) (Luc MartinModif) [Supprimer](#)

Utilisateur [usertest](#) (Testeur Test) [Supprimer](#)



[Gestion des voitures](#) [Gestion des utilisateurs](#) [Gestion des trajets](#)

L'utilisateur a bien été créé !

Utilisateur [dupontj](#) (Jean Dupont) [Supprimer](#)

Utilisateur [durandm](#) (Marie Durand) [Supprimer](#)

Utilisateur [martinluc](#) (Luc MartinModif) [Supprimer](#)

Utilisateur [nouveau_user](#) (Test Nouveau) [Supprimer](#)

Section 4 : Contrôleur et Modèle Trajet

Pour tester notre modèle générique sur une autre table, on ajoute l'entité Trajet. On crée ModelTrajet.php qui hérite de Model avec static::\$object = 'trajet' et static::\$primary = 'id'. On crée aussi ControllerTrajet.php avec readAll(), read() et delete(), ainsi que les vues dans view/trajet/. Tout marche directement sans avoir besoin d'écrire de code SQL.

```
C:\xampp\htdocs\TD6\model\ModelTrajet.php - Notepad++
Fichier Édition Recherche Affichage Encodage Langage Paramètres Outils Macro Exécution Modules d'extension Documents ?

ControllerVoiture.php routeur.php update.php update.php ModelTrajet.php x
1
2 require_once File::build_path(array("model", "Model.php"));
3
4 class ModelTrajet extends Model {
5     protected static $object = 'trajet';
6     protected static $primary = 'id';
7
8     private $id;
9     private $depart;
10    private $arrivee;
11    private $date;
12    private $nbplaces;
13    private $prix;
14    private $conducteur_login;
15
16    public function __construct($id = NULL, $depart = NULL, $arrivee = NULL, $date = NULL, $nb = NULL, $prix = NULL, $conducteur_login = NULL) {
17        if (!is_null($id)) {
18            $this->id = $id;
19            $this->depart = $depart;
20            $this->arrivee = $arrivee;
```



3) Conclusion

En conclusion, ce TD nous a permis d'améliorer notre architecture MVC en évitant de répéter le même code partout. Grâce à la classe mère Model et à l'héritage, on a pu rendre toutes les méthodes CRUD (selectAll, select, delete, save, update) génériques pour toutes nos tables. On a aussi simplifié nos formulaires en fusionnant create et update, et on a un routeur qui peut gérer plusieurs contrôleurs facilement. Notre projet est maintenant beaucoup plus propre et bien plus simple à continuer.